| | |
|---|---|
| | SR UNIVERSITY<br>Campus Warangal |
| | Program: II - B. Tech (CS& AI) Batch: |
| | Name:N.KRUTHIKA Reg. No:2503A52584 |
| | Department: Computer Science and AI Semester: I |
| | AI Assisted Coding-Assignment Date: |

## TASK 1 – Improve Readability

### 1. Original Code

```
a = 80
b = 70
c = 90
x = (a + b + c) / 3
if x >= 50:
    print("pass")
else:
    print("fail")
```

### 2. AI Prompt

Refactor this Python code to improve readability. Use meaningful variable names, create a calculate_average() function, preserve the original pass/fail rule, and provide test output.

### 3. AI Suggestion

Use descriptive names for the marks and average, move the calculation into a function, and keep the pass mark of 50 unchanged.

### 4. Refactored Code

```
def calculate_average(mark1, mark2, mark3):
    return (mark1 + mark2 + mark3) / 3

average_mark = calculate_average(80, 70, 90)
if average_mark >= 50:
```

```
    print("pass")
else:
    print("fail")
```

### 5. Test Output

```
pass
Average = 80.0
```

### 6. Explanation

The original average is (80 + 70 + 90) / 3 = 80.0, so both versions print "pass". The function makes the calculation reusable, meaningful names make the code easier to understand, and consistent indentation improves readability.

Accepted suggestion: meaningful names and a reusable function. Rejected/modified suggestion: none was necessary; the original threshold and output wording were retained to preserve behavior.

## TASK 2 – Remove Repeated Code

### 1. Original Code

```
name1 = "Arun"
marks1 = 85
if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 = "Priya"
marks2 = 72
if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

## 2. AI Prompt

Identify duplicated pass/fail logic and refactor it into a reusable function. Test it with at least four students and preserve the rule that marks >= 50 means Pass.

## 3. AI Suggestion

Create a display_result() function that accepts a student's name and marks. Call it for each student rather than repeating the conditional.

## 4. Refactored Code

```
def display_result(student_name, marks):
    if marks >= 50:
        result = "Pass"
    else:
        result = "Fail"
    print(student_name, result)

students = [("Arun", 85), ("Priya", 72), ("Kiran", 42), ("Meena", 50)]
for student_name, marks in students:
    display_result(student_name, marks)
```

## 5. Test Output

```
Arun Pass
Priya Pass
Kiran Fail
Meena Pass
```

## 6. Explanation

The function is written once and reused for all students. This avoids duplicated code and means that if the pass rule changes, it only needs to be updated in one place. Four cases were tested, including a failing mark and the boundary value 50.

Accepted suggestion: reusable function and a loop over student records. Rejected/modified suggestion: none; the threshold of 50 was preserved.

## TASK 3 – Refactor a Long Conditional

### 1. Original Code

```
age = 68
temperature = 39

if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

### 2. AI Prompt

Simplify this conditional by creating determine_priority(age, temperature). Preserve exactly the original rules and test at least four age/temperature combinations.

### 3. AI Suggestion

The conditions can be simplified: high priority when both age >= 60 and temperature >= 38; medium priority when exactly one of those conditions is true; otherwise normal priority.

### 4. Refactored Code

```
def determine_priority(age, temperature):
    is_older_adult = age >= 60
    has_high_temperature = temperature >= 38

    if is_older_adult and has_high_temperature:
        return "High Priority"
    elif is_older_adult or has_high_temperature:
        return "Medium Priority"
    else:
        return "Normal Priority"
```

```
cases = [(68, 39), (68, 37), (45, 39), (45, 37)]
for age, temperature in cases:
    print(age, temperature, "->", determine_priority(age, temperature))
```

### 5. Test Output

```
68 39 -> High Priority
68 37 -> Medium Priority
45 39 -> Medium Priority
45 37 -> Normal Priority
```

### 6. Explanation

The simplified function preserves the original behavior for the four combinations: both conditions true gives High Priority; only one true gives Medium Priority; neither true gives Normal Priority. Named Boolean variables make the meaning of each condition clearer.

Accepted suggestion: combine the two medium-priority cases using `or`. Rejected/modified suggestion: do not change the age or temperature thresholds, because that would alter the assignment's behavior.

## TASK 4 – Refactor Inefficient List Processing

### 1. Original Code

```
patient_ids = [101, 205, 309, 412, 518, 623]
target = 412
found = False

for patient_id in patient_ids:
    if patient_id == target:
        found = True

if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

## 2. AI Prompt

Improve readability of this patient ID lookup. Explain whether a list or set is better for frequent membership checks, implement the justified option, and test both present and missing IDs.

## 3. AI Suggestion

Use the `in` operator for a clear membership test. If the collection is used for frequent lookups and duplicate IDs/order are not important, a set is generally better on average. Keep a list if order or duplicates matter.

## 4. Refactored Code

```
patient_ids = {101, 205, 309, 412, 518, 623}  # set for frequent lookups

def find_patient(patient_id):
    if patient_id in patient_ids:
        print("Patient Found")
    else:
        print("Patient Not Found")

find_patient(412)
find_patient(999)
```

## 5. Test Output

```
Patient Found
Patient Not Found
```

## 6. Explanation

The `in` operator is more readable than manually setting a flag in a loop. Typical list membership lookup takes O(n) time because elements may be checked one by one. Set membership is O(1) on average, though worst-case behavior can be slower. A set is suitable here because the task only checks membership; use a list if order or duplicates must be preserved.

Accepted suggestion: set membership for frequent lookups. Modified suggestion: this choice assumes ordering and duplicate IDs are not required by the application.

## TASK 5 – Complete Legacy Code Review with AI

### 1. Original Code

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg)
```

### 2. AI Prompt

Review and refactor this legacy Python code. Identify at least five improvements, use meaningful names and reusable functions, preserve the pass threshold and output behavior, and verify the results with tests. List accepted or rejected suggestions with reasons.

### 3. AI Suggestions (at least five)

1. Use descriptive names such as student_name, marks, total_marks, and average_marks.
2. Create a function to determine and display each student's pass/fail result.
3. Create a function to calculate the average marks.
4. Use tuple unpacking (`for student_name, marks in students`) instead of numeric indexes such as s[0] and s[1].
5. Use `sum()` with a generator expression to calculate the total concisely.
6. Keep the pass threshold of 50 unchanged to preserve behavior.

7. Consider validating that the students list is not empty before dividing by its length.

## 4. Refactored Code

```
def get_result(marks):
    return "Pass" if marks >= 50 else "Fail"

def calculate_average(students):
    if not students:
        return 0
    total_marks = sum(marks for student_name, marks in students)
    return total_marks / len(students)

def display_student_results(students):
    for student_name, marks in students:
        print(student_name, get_result(marks))

students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]
display_student_results(students)
average_marks = calculate_average(students)
print("Average =", average_marks)
```

## 5. Test Output

```
Arun Pass
Priya Fail
Rahul Pass
Meena Pass
Average = 65.75
```

## 6. Explanation and Comparison

Total marks = 78 + 45 + 88 + 52 = 263. Average = 263 / 4 = 65.75. Both original and refactored code print the same student results and average for the given data. The empty-list guard is an additional safety improvement; for a non-empty list, it does not change the original output.

### 7. Accepted / Rejected Suggestions

Accepted: descriptive names, reusable functions, tuple unpacking, and sum() for concise total calculation. These changes improve clarity and reduce repeated logic.

Modified: the empty-list case returns 0. The original code would raise ZeroDivisionError for an empty list, so this is an intentional safety improvement for an input not present in the original example.

Rejected: changing the pass mark or changing the output labels was rejected because it would alter the intended behavior.

### 8. Most Useful Refactoring Change

The most useful change is separating the logic into reusable functions. Each function has one clear responsibility, making the code easier to test, maintain, and reuse. If the pass rule changes, it can be updated in one place.

## Overall Conclusion

Code refactoring improves readability, reduces duplication, and makes programs easier to maintain without changing intended behavior. AI suggestions are useful, but every change should be reviewed and tested. The refactored versions in these tasks preserve the specified rules and produce the expected outputs for the test cases.